

# MicroProfile Telemetry w piksa-1

## Co to jest MicroProfile Telemetry

Eclipse MicroProfile Telemetry (od MP 5.0, 2022) standaryzuje integrację **OpenTelemetry** w aplikacjach MicroProfile. Obejmuje trzy filary:

| Filar                      | Opis                                                                                                         |
|----------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Distributed Tracing</b> | Każde żądanie HTTP dostaje "trace" złożony ze "spanów" — jednostek pracy z czasem, statusem i atrybutami.    |
| <b>Context Propagation</b> | Nagłówki W3C TraceContext przekazywane między serwisami automatycznie, umożliwiając pełną ścieżkę wywołania. |
| <b>Baggage</b>             | Dowolne pary klucz-wartość podróżujące razem z kontekstem tracing.                                           |

### Typowe zastosowania:

- Diagnostowanie opóźnień — widać, gdzie żądanie spędza czas (HTTP, DB, zewnętrzne API)
- Debugging błędów produkcyjnych bez reprodukcji lokalnej
- Obserwacja stanu sesji i zachowań użytkowników w czasie rzeczywistym
- W architekturze mikroservisów: pełna ścieżka przez wiele serwisów w jednym widoku

### Kluczowe API (używane w kodzie aplikacji):

| Element                              | Gdzie           | Efekt                                                           |
|--------------------------------------|-----------------|-----------------------------------------------------------------|
| <code>@WithSpan("nazwa")</code>      | metoda CDI      | tworzy nowy <i>child span</i> obejmujący wywołanie metody       |
| <code>@SpanAttribute("klucz")</code> | parametr metody | dodaje wartość parametru jako atrybut do spana                  |
| <code>@Inject Span</code>            | pole klasy      | dostęp do <i>bieżącego</i> spana (do dodania atrybutów/eventów) |

W Quarkus specyfikację implementuje `quarkus-opentelemetry`.

## Proponowane zmiany w piksa-1

### Motywacja

- JAX-RS endpoints (`/hello`, `/items`) są **już automatycznie tracowane** po dodaniu `quarkus-opentelemetry` — żadnego kodu nie trzeba pisać dla warstwy boundary.
- Warstwa `ItemController` (control) jest **niewidoczna** w domyślnych tracach — `@WithSpan` ją ujawnia jako child span.

- Licznik sesji w `GreetingResource` jest **ciekawą daną biznesową** — warto ją dołączyć do aktualnego spana przez `@Inject Span`.

---

## Zmiana 1 — `pom.xml`

Dodać dependency (wersja zarządzana przez Quarkus BOM):

```
<dependency>
  <groupId>io.quarkus</groupId>
  <artifactId>quarkus-opentelemetry</artifactId>
</dependency>
```

---

## Zmiana 2 — `application.properties`

```
# --- OpenTelemetry / MicroProfile Telemetry ---
quarkus.application.name=piksa-l
quarkus.otel.exporter.otlp.endpoint=http://localhost:4317

# Wyłącz w testach (brak kolektora w środowisku CI/test)
%test.quarkus.otel.sdk.disabled=true
```

`quarkus.otel.exporter.otlp.endpoint` wskazuje na lokalny OTLP collector (np. Jaeger all-in-one, Grafana Alloy, OpenTelemetry Collector).

Profil `%test` wyłącza SDK — testy nie zmieniają zachowania i nie wymagają kolektora.

---

## Zmiana 3 — `ItemController.java`

Dodać `@WithSpan` i `@SpanAttribute` — child span dla warstwy control:

```
import io.opentelemetry.instrumentation.annotations.SpanAttribute;
import io.opentelemetry.instrumentation.annotations.WithSpan;

@WithSpan("item.create")
public Item createItem(
    @SpanAttribute("item.name") String name,
    @SpanAttribute("item.description") String description) {
    // ...
}
```

**Efekt w Jaeger/Grafana Tempo:** trace dla `POST /items` pokazuje zagnieżdżony span `item.create` z atrybutami `item.name` i `item.description`. Widoczna jest granica między warstwą boundary a control.

---

## Zmiana 4 — `GreetingResource.java`

Wstrzyknąć `Span` i wzbogacić istniejący HTTP span o atrybut biznesowy:

```
import io.opentelemetry.api.trace.Span;

// pole klasy - wstrzykiwany jest bieżący span (HTTP span auto-stworzony przez RESTEasy)
@Inject
Span span;

@GET
@Produces(MediaType.TEXT_PLAIN)
public String hello() {
    final var count = userSession.incrementAndGet();
    span.setAttribute("session.visitCount", count);
    return "Hello from Quarkus REST (visit #" + count + " in this session)";
}
```

**Efekt:** span dla `GET /hello` zawiera atrybut `session.visitCount`, co pozwala filtrować i analizować ruch per-sesja w narzędziu observability bez zmiany formatu odpowiedzi HTTP.

## Podsumowanie zmian

| Plik                                                                       | Rodzaj zmiany                                           |
|----------------------------------------------------------------------------|---------------------------------------------------------|
| <code>pom.xml</code>                                                       | +1 dependency ( <code>quarkus-opentelemetry</code> )    |
| <code>src/main/resources/application.properties</code>                     | +3 właściwości OTel                                     |
| <code>src/main/java/edu/san/item/control/ItemController.java</code>        | <code>@WithSpan</code> + <code>@SpanAttribute</code>    |
| <code>src/main/java/edu/san/greeting/boundary/GreetingResource.java</code> | <code>@Inject Span</code> + <code>setAttribute()</code> |

Pliki testowe **nie wymagają zmian** — `%test.quarkus.otel.sdk.disabled=true` wyłącza OTel w testach transparentnie.

## Weryfikacja

```
# Testy powinny przechodzić bez zmian
./mvnw test

# Uruchomić Jaeger lokalnie (obserwacja traces)
docker run --rm -p 16686:16686 -p 4317:4317 jaegertracing/all-in-one

# Uruchomić aplikację i wywołać endpointy
./mvnw quarkus:dev
curl http://localhost:8080/hello
```

```
curl -X POST http://localhost:8080/items \  
  -H "Content-Type: application/json" \  
  -d '{"name":"Widget","description":"test"}'
```

# Traces widoczne w Jaeger UI

open <http://localhost:16686>